Software Design & Software Engineering

Mid-Semester Comprehensive Notes

Topics Covered

- Software Engineering
- Object-Oriented Programming
- UML Diagrams
- Entity-Relationship Diagrams
- Design Patterns

Compiled for Mid-Semester Examination Preparation

Contents

1 Software Engineering **3**
- 1.1 Software Lifecycle . . . 3
  - 1.1.1 Phases of the Software Lifecycle . . . 3
  - 1.1.2 Common SDLC Models . . . 4
- 1.2 Quality Attributes . . . 4
  - 1.2.1 Key Quality Attributes . . . 4
  - 1.2.2 Critical Quality Attributes in Depth . . . 5

2 Object-Oriented Programming (OOP) **6**
- 2.1 Core Concepts . . . 6
  - 2.1.1 Classes & Objects . . . 6
  - 2.1.2 Constructors . . . 7
  - 2.1.3 Invariants . . . 8
- 2.2 OOP Mechanisms . . . 8
  - 2.2.1 Encapsulation . . . 9
  - 2.2.2 Abstraction . . . 10
  - 2.2.3 Inheritance . . . 12
  - 2.2.4 Polymorphism . . . 13
- 2.3 Object Relationships & Design Concepts . . . 14
  - 2.3.1 Composition . . . 14
  - 2.3.2 Data Transfer Object (DTO) and Domain Object . . . 15
  - 2.3.3 Observability . . . 16

3 Unified Modelling Language (UML) **17**
- 3.1 Structural UML Diagrams . . . 17
  - 3.1.1 Class Diagram . . . 17
  - 3.1.2 Class Relationships . . . 18
- 3.2 Behavioural UML Diagrams . . . 18
  - 3.2.1 Sequence Diagram . . . 18
  - 3.2.2 Use Case Diagram . . . 19

4 Entity Relationship Diagram (ERD) **20**
- 4.1 Components . . . 20
  - 4.1.1 Entities . . . 20
  - 4.1.2 Attributes . . . 20
  - 4.1.3 Relationships . . . 21
- 4.2 Mapping Cardinalities . . . 21
- 4.3 Participation Constraints . . . 22
- 4.4 ERD Notations . . . 22
  - 4.4.1 Chen Notation . . . 22
  - 4.4.2 Crow's Foot Notation . . . 22

5 Design Patterns **24**
- 5.1 Singleton Design Pattern . . . 24
  - 5.1.1 Key Characteristics . . . 24
  - 5.1.2 Implementations . . . 24

5.1.3 Singleton: Advantages & Disadvantages 26
5.2 Factory Design Pattern 26
5.2.1 Simple Factory (Utility / Static Factory) 26
5.2.2 Factory Method Pattern (GoF) 27
5.2.3 Abstract Factory Pattern 28
5.2.4 Factory Pattern: Advantages & Disadvantages 29

Quick Revision Summary **31**

1 Software Engineering

1.1 Software Lifecycle

Definition

The **Software Development Lifecycle (SDLC)** is a structured process used by software teams to plan, design, build, test, and deliver software. It defines distinct phases, their order, and the activities performed in each phase.

1.1.1 Phases of the Software Lifecycle

Core SDLC Phases

1. **Requirements Gathering & Analysis**
   Identify stakeholder needs. Produce a Software Requirements Specification (SRS). Classify requirements as *functional* (what the system does) or *non-functional* (how well it does it).

2. **System Design**
   Translate requirements into a blueprint. Includes high-level architecture (HLD) and detailed module design (LLD). Choose data models, technologies, interfaces, and security protocols.

3. **Implementation (Coding)**
   Developers write source code following the design documents. Coding standards and version control are applied here.

4. **Testing & Quality Assurance**
   Verify the system against requirements. Includes unit testing, integration testing, system testing, and user acceptance testing (UAT).

5. **Deployment**
   Release the software to a production environment. May involve CI/CD pipelines, blue-green deployments, or canary releases.

6. **Maintenance & Evolution**
   Bug fixes, patches, performance improvements, and feature enhancements post-deployment. Typically the longest and costliest phase.

1.1.2 Common SDLC Models

Model	Key Characteristics
Waterfall	Sequential, phase-gated; each phase must complete before the next begins. Simple but rigid.
Iterative	Repeat cycles of design-build-test; improve each iteration.
Agile	Short sprints (1–4 weeks); continuous feedback; working software over documentation.
Spiral	Risk-driven; combines iterative with explicit risk analysis at each loop.
V-Model	Each development phase paired with a corresponding testing phase (Verification & Validation).
DevOps	Integrates development and operations; emphasises automation, CI/CD, and monitoring.

Note

Why does the SDLC matter? Without a defined lifecycle, projects suffer from scope creep, budget overruns, and poor quality. The SDLC provides checkpoints, documentation standards, and accountability at every stage.

1.2 Quality Attributes

Definition

Quality Attributes (also called *non-functional requirements* or *system qualities*) describe *how well* a system performs its functions, rather than *what* it does. They profoundly influence architectural decisions.

1.2.1 Key Quality Attributes

The FURPS+ Model of Quality Attributes

Functionality
: Correctness, security, compliance — does the software do what it is supposed to do?

Usability
: Ease of learning, efficiency of use, memorability, error prevention, aesthetics.

Reliability
: Availability (uptime), mean time between failures (MTBF), fault tolerance, recoverability.

Performance
: Response time, throughput, latency, resource utilisation (CPU, memory, bandwidth).

Supportability
: Maintainability, testability, adaptability, scalability, portability.

+ (Plus)
Design constraints, implementation constraints, interface constraints, physical constraints.

1.2.2 Critical Quality Attributes in Depth

1. **Maintainability** — ease of modifying the system to fix bugs, improve performance, or adapt to a changing environment.
   - Sub-qualities: modularity, reusability, analysability, modifiability, testability.
   - Measured by: cyclomatic complexity, coupling/cohesion metrics.
2. **Scalability** — ability to handle growing amounts of work or to expand to accommodate growth.
   - *Vertical scaling*: add more resources (CPU, RAM) to a single node.
   - *Horizontal scaling*: add more nodes to distribute load.
3. **Security** — protection against unauthorised access, data breaches, and attacks.
   - Principles: Confidentiality, Integrity, Availability (CIA Triad).
   - Mechanisms: authentication, authorisation, encryption, auditing.
4. **Availability** — the fraction of time a system is operational.

$$\text{Availability} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

   where MTBF = Mean Time Between Failures, MTTR = Mean Time To Repair.
5. **Portability** — ability to transfer the software from one environment to another (OS, hardware, cloud provider).
6. **Interoperability** — ability of the system to work with other systems or products without special effort.

Note

Quality attributes often **trade off** against each other. For example, maximising security can reduce performance; optimising for availability may increase cost. Architects must balance these tensions explicitly.

2 Object-Oriented Programming (OOP)

2.1 Core Concepts

2.1.1 Classes & Objects

Definition

A **class** is a blueprint or template that defines the structure (attributes) and behaviour (methods) of objects. An **object** is a concrete instance of a class with its own specific state.

Listing 1: Class and Object in Java

```java
// Class definition -- blueprint
public class BankAccount {
    // Attributes (fields / instance variables)
    private String accountNumber;
    private double balance;
    private String owner;

    // Constructor
    public BankAccount(String accountNumber, String owner, double
        initialBalance) {
        this.accountNumber = accountNumber;
        this.owner = owner;
        this.balance = initialBalance;
    }

    // Method (behaviour)
    public void deposit(double amount) {
        this.balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}

// Creating objects (instances)
BankAccount acc1 = new BankAccount("ACC001", "Alice", 5000.00);
BankAccount acc2 = new BankAccount("ACC002", "Bob",   1200.00);
// acc1 and acc2 are distinct objects sharing the same class
   structure
```

Class Anatomy

Component	Description
Fields / Attributes	Variables that store the object's state
Methods	Functions defining behaviour
Constructor	Special method called when an object is created
Access modifiers	`public`, `private`, `protected`, package-private
Static members	Belong to the class, not individual instances

2.1.2 Constructors

Definition

A **constructor** is a special method invoked automatically when an object is instantiated. It has the same name as the class and no return type. Its purpose is to *initialise the object to a valid state.*

Listing 2: Types of Constructors

```java
public class Student {
    private String name;
    private int age;
    private double gpa;

    // 1. Default Constructor (no arguments)
    public Student() {
        this.name = "Unknown";
        this.age  = 0;
        this.gpa  = 0.0;
    }

    // 2. Parameterised Constructor
    public Student(String name, int age, double gpa) {
        this.name = name;
        this.age  = age;
        this.gpa  = gpa;
    }

    // 3. Copy Constructor
    public Student(Student other) {
        this.name = other.name;
        this.age  = other.age;
        this.gpa  = other.gpa;
    }
}
```

- **Constructor Overloading**: multiple constructors with different parameter lists in the same class.
- **`this()` call**: one constructor may call another constructor of the same class.

- `super()` **call**: a subclass constructor calls the parent class constructor.
- If no constructor is defined, Java provides a *default constructor* automatically (only if no other constructors exist).

2.1.3 Invariants

Definition

A **class invariant** is a condition or constraint that must always be true for every instance of the class throughout its lifetime — before and after every method call. Invariants define the *valid state space* of an object.

Example

For a `BankAccount`, a reasonable invariant might be: *"The balance must never be negative"* — i.e., `balance` ≥ 0.

Listing 3: Enforcing Invariants

```java
public class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        if (initialBalance < 0)
            throw new IllegalArgumentException("Balance cannot be negative.");
        this.balance = initialBalance;    // invariant established in constructor
    }

    public void withdraw(double amount) {
        if (amount > balance)
            throw new IllegalStateException("Insufficient funds.");
        this.balance -= amount;           // invariant maintained after withdraw
    }
}
```

Note

Invariants are closely related to **Design by Contract**: every method has *preconditions* (what must be true before it runs), *postconditions* (what is guaranteed after it runs), and the object's *invariant* (what is always true).

2.2 OOP Mechanisms

2.2.1 Encapsulation

Definition

Encapsulation is the bundling of data (fields) and the methods that operate on that data into a single unit (class), while *restricting direct access* to some of the object's components. It protects the internal state and hides implementation details.

Benefits of Encapsulation:

- Protects object integrity by preventing external code from setting invalid state.
- Reduces coupling between classes — callers depend on the interface, not the implementation.
- Allows internal implementation to change without affecting external users.
- Promotes modularity and easier maintenance.

Getters and Setters

Definition

Getters (accessors) are public methods that return the value of private fields. **Setters** (mutators) are public methods that update a private field, optionally enforcing validation logic.

Listing 4: Encapsulation with Getters and Setters

```java
public class Person {
    private String name;    // private -- not directly accessible
    private int age;

    public Person(String name, int age) {
        this.name = name;
        setAge(age);        // use setter for validation
    }

    // Getter
    public String getName() { return name; }

    // Setter with validation (invariant enforcement)
    public void setAge(int age) {
        if (age < 0 || age > 150)
            throw new IllegalArgumentException("Invalid age: " +
                age);
        this.age = age;
    }

    public int getAge() { return age; }
}
```

Note

Not every field needs a setter. **Immutable objects** only expose getters; the state is set once in the constructor and never changes. This is a powerful design choice (e.g., Java's `String`, `LocalDate`).

2.2.2 Abstraction

Definition

Abstraction is the process of hiding implementation complexity and exposing only the essential features of an object. It focuses on *what* an entity does rather than *how* it does it.

Abstract Classes

Definition

An **abstract class** is a class that *cannot be instantiated directly*. It may contain both abstract methods (no body, must be overridden by subclasses) and concrete methods (with implementation). It serves as a partial template.

Listing 5: Abstract Class Example

```java
public abstract class Shape {
    private String colour;

    public Shape(String colour) { this.colour = colour; }

    // Abstract method -- subclasses MUST implement this
    public abstract double area();
    public abstract double perimeter();

    // Concrete method -- shared implementation
    public void printInfo() {
        System.out.println("Shape: " + getClass().getSimpleName()
            + " | Colour: " + colour
            + " | Area: " + area());
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(String colour, double radius) {
        super(colour);
        this.radius = radius;
    }

    @Override
    public double area() { return Math.PI * radius * radius; }

```

```java
    @Override
    public double perimeter() { return 2 * Math.PI * radius; }
}
```

Interfaces

Definition

An **interface** is a purely abstract type that defines a *contract* — a set of method signatures (and optionally default/static methods) that a class must implement. A class can implement multiple interfaces (achieving a form of multiple inheritance).

Listing 6: Interface and Contract Example

```java
// Interface defines the contract
public interface Printable {
    void print();                    // implicitly public abstract
    default void printTwice() {      // default method (Java 8+)
        print(); print();
    }
}

public interface Serialisable {
    String serialise();
    void deserialise(String data);
}

// A class can implement multiple interfaces
public class Invoice implements Printable, Serialisable {
    private String content;

    @Override
    public void print() { System.out.println(content); }

    @Override
    public String serialise() { return content; }

    @Override
    public void deserialise(String data) { this.content = data; }
}
```

Feature	Abstract Class	Interface
Instantiation	Cannot instantiate	Cannot instantiate
Constructor	Can have constructors	Cannot have constructors
Fields	Can have instance fields	Only `static final` constants
Method types	Abstract + concrete	Abstract + default + static
Inheritance	Single (extends)	Multiple (implements)
Use case	Shared base with partial impl.	Define a capability / role

Contracts A **contract** in OOP (especially via interfaces) specifies:

- **Preconditions** — what the caller guarantees before calling a method.
- **Postconditions** — what the method guarantees upon return.
- **Invariants** — conditions always maintained by the implementing class.

2.2.3 Inheritance

Definition

Inheritance allows a class (*subclass* / child) to derive properties and behaviours from another class (*superclass* / parent), enabling *code reuse* and establishing an **IS-A** relationship.

Listing 7: Inheritance in Java

```java
public class Animal {
    protected String name;
    protected int age;

    public Animal(String name, int age) {
        this.name = name; this.age = age;
    }

    public void eat() { System.out.println(name + " is eating."); }
    public String describe() { return name + " (age " + age + ")"; }
}

public class Dog extends Animal {
    private String breed;

    public Dog(String name, int age, String breed) {
        super(name, age);        // call parent constructor
        this.breed = breed;
    }

    public void bark() { System.out.println(name + " says: Woof!"); }

    @Override
    public String describe() {
        return super.describe() + " | Breed: " + breed;
    }
}
```

Inheritance Key Points

- **Method Overriding**: subclass provides a specific implementation of a superclass method (`@Override`).

- `super`: used to call parent class constructor or methods.
- **Access**: `private` members of parent are *not* accessible in child (use `protected`).
- `final` **class**: cannot be subclassed.
- **Java**: single inheritance for classes; multiple inheritance via interfaces.
- **Liskov Substitution Principle (LSP)**: a subclass object must be usable wherever a parent object is expected.

2.2.4 Polymorphism

Definition

Polymorphism (Greek: *many forms*) allows a single interface or method name to work with objects of different types. The correct method is selected based on the actual type of the object at runtime (*dynamic dispatch*).

Listing 8: Runtime Polymorphism (Method Overriding)

```java
public class Shape {
    public double area() { return 0; }
}

public class Circle extends Shape {
    private double r;
    public Circle(double r) { this.r = r; }
    @Override public double area() { return Math.PI * r * r; }
}

public class Rectangle extends Shape {
    private double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h
        ; }
    @Override public double area() { return w * h; }
}

// Polymorphic usage
Shape[] shapes = { new Circle(5), new Rectangle(4, 6) };
for (Shape s : shapes) {
    System.out.println(s.area()); // correct method called at
        runtime
}
```

Listing 9: Compile-Time Polymorphism (Method Overloading)

```java
public class Calculator {
    public int add(int a, int b)          { return a + b; }
    public double add(double a, double b){ return a + b; }
    public int add(int a, int b, int c)  { return a + b + c; }
```

```
}
```

Type	Mechanism	Resolution
Compile-time	Method overloading	At compile time
Runtime	Method overriding + inheritance	At runtime (dynamic dispatch)

2.3 Object Relationships & Design Concepts

2.3.1 Composition

Definition

Composition is a **HAS-A** relationship where an object is made up of one or more objects of other classes. The component objects *do not exist independently* of the containing object — if the parent is destroyed, so are its components (strong ownership).

Listing 10: Composition vs Aggregation

```java
// Composition: Engine cannot exist without Car
public class Engine {
    private int horsepower;
    public Engine(int hp) { this.horsepower = hp; }
}

public class Car {
    private Engine engine;    // Car "owns" Engine
    private String model;

    public Car(String model, int hp) {
        this.model  = model;
        this.engine = new Engine(hp); // Engine created inside Car
    }
}

// Aggregation: Department can exist without University
public class Department {
    private String name;
    public Department(String name) { this.name = name; }
}

public class University {
    private List<Department> departments; // University references Departments
    public University(List<Department> depts) {
        this.departments = depts;         // Departments exist independently
    }
}
```

	Composition	Aggregation
Relationship	Strong HAS-A	Weak HAS-A
Lifetime	Component tied to parent	Component independent
Dependency	Parent creates component	Parent holds reference
UML	Filled diamond	Open diamond

Note

Favour Composition over Inheritance — a widely-accepted design principle. Composition is more flexible; inheritance creates tight coupling between parent and child.

2.3.2 Data Transfer Object (DTO) and Domain Object

Definition

A **Data Transfer Object (DTO)** is a plain object used to carry data between layers or services. It has no business logic — only fields, getters, setters, and possibly a constructor. It is used to decouple the internal domain model from the external API.

Definition

A **Domain Object** (Entity) is a rich object that contains both data *and* business logic related to a real-world concept (e.g., `Order`, `Product`, `Customer`).

Listing 11: DTO vs Domain Object

```java
// Domain Object -- contains business logic
public class Order {
    private List<OrderLine> lines;
    private OrderStatus status;

    public void addItem(Product p, int qty) { /* business rules */ }
    public Money totalPrice() { /* computation logic */ }
    public void cancel() {
        if (status == OrderStatus.SHIPPED)
            throw new IllegalStateException("Cannot cancel shipped order.");
        this.status = OrderStatus.CANCELLED;
    }
}

// DTO -- flat, no logic, used in API layer
public class OrderDTO {
    public String orderId;
    public String customerName;
    public double totalAmount;
    public String status;
}
```

When to Use a DTO

- Sending data over a network (REST API response/request body).
- Crossing architectural boundaries (e.g., service layer to presentation layer).
- Avoiding the exposure of internal domain structure.
- Shaping data for a specific use-case (aggregating multiple domain fields).

2.3.3 Observability

Definition

Observability is the ability to understand the internal state of a system by examining its external outputs. In OOP and distributed systems, the three pillars of observability are: **Logs**, **Metrics**, and **Traces**.

Three Pillars of Observability

Logs	Timestamped records of discrete events (errors, state changes, user actions). Provide textual context. Examples: application logs, audit logs.
Metrics	Numerical measurements over time (counters, gauges, histograms). Examples: CPU usage, request count per second, memory utilisation.
Traces	End-to-end tracking of a request through multiple services/components. Reveal latency bottlenecks. Used in microservices with tools like Jaeger, Zipkin.

In OOP design, observability is achieved by instrumenting objects to emit events, expose metrics, and support tracing — often via the **Observer Design Pattern**, structured logging frameworks (SLF4J, Log4j), and APM tools.

3 Unified Modelling Language (UML)

Definition

UML (Unified Modelling Language) is a standardised visual language for describing, visualising, and documenting the artefacts of a software system. It is maintained by the Object Management Group (OMG).

UML diagrams are classified into two broad categories:

- **Structural diagrams** — depict the static structure (classes, components, objects).
- **Behavioural diagrams** — depict dynamic behaviour (interactions, state changes, use cases).

3.1 Structural UML Diagrams

3.1.1 Class Diagram

Definition

A **Class Diagram** is the most widely used UML diagram. It shows the *classes* in a system, their *attributes* and *operations*, and the *relationships* between classes.

Class Notation A class is drawn as a rectangle divided into three compartments:

ClassName

`- attribute1 : Type`
`- attribute2 : Type`
`+ attribute3 : Type`

`+ method1() : ReturnType`
`+ method2(p : Type) : void`
`- helperMethod() : int`

Visibility Modifiers:

- `+` Public
- `-` Private
- `#` Protected
- `~` Package (default)

3.1.2 Class Relationships

UML Class Relationships Summary

Relationship	Description	UML Notation
Association	General link between classes	Solid line
Directed Assoc.	One class uses another	Solid line + open arrow
Aggregation	Weak HAS-A (whole-part)	Solid line + open diamond
Composition	Strong HAS-A (owned part)	Solid line + filled diamond
Inheritance	IS-A (generalisation)	Solid line + hollow triangle
Realisation	Class implements interface	Dashed line + hollow triangle
Dependency	One class depends on another	Dashed line + open arrow

Multiplicity Notation on Associations:

- `1` — exactly one
- `0..1` — zero or one
- `*` or `0..*` — zero or more
- `1..*` — one or more
- `m..n` — from m to n

Example

Reading a relationship: In a library system, a `Library` *composes* many `Book` objects (`1` Library to `1..*` Books with filled diamond). A `Book` *is associated* with `0..1 Author` (association). A `EBook` *inherits* from `Book` (hollow triangle pointing to `Book`).

3.2 Behavioural UML Diagrams

3.2.1 Sequence Diagram

Definition

A **Sequence Diagram** is a behavioural UML diagram that shows *how objects interact in a time-ordered sequence*. It depicts message passing between participants (actors, objects, systems) along a vertical time axis.

Key Elements:

- **Lifeline**: A vertical dashed line representing an object's existence over time, with a box at the top showing the object name.
- **Activation bar**: A narrow rectangle on the lifeline indicating when an object is active (processing).

- **Messages**: Horizontal arrows between lifelines representing method calls or signals.
  - *Synchronous call*: solid arrowhead (caller waits).
  - *Return*: dashed arrow with open arrowhead.
  - *Asynchronous*: solid arrowhead without blocking.
- **Combined Fragments**: `alt` (if/else), `loop`, `opt` (optional), `par` (parallel).

Example

Login Sequence:

```
User → LoginController: login(username, password)
LoginController → AuthService: authenticate(username, password)
AuthService → UserRepository: findByUsername(username)
UserRepository --→ AuthService: User
AuthService --→ LoginController: AuthToken
LoginController --→ User: 200 OK + Token
```

3.2.2 Use Case Diagram

Definition

A **Use Case Diagram** shows the *functional requirements* of a system from the user's perspective. It captures what a system does (use cases) and who interacts with it (actors), without detailing how it is done.

Key Elements:

- **Actor**: An external entity (person, system, device) that interacts with the system. Drawn as a stick figure.
- **Use Case**: A specific function or goal provided by the system. Drawn as an oval.
- **System Boundary**: A rectangle enclosing all use cases representing the scope of the system.
- **Associations**: Lines connecting actors to the use cases they participate in.
- ⟨⟨**include**⟩⟩: A use case always includes another (mandatory inclusion).
- ⟨⟨**extend**⟩⟩: A use case conditionally extends another (optional behaviour).
- **Generalisation**: An actor or use case inherits from another.

Example

Online Shopping System:

Actors: Customer, Admin, Payment Gateway
Use Cases: Browse Products, Add to Cart, Checkout, Make Payment (⟨⟨include⟩⟩ for Checkout), Apply Coupon (⟨⟨extend⟩⟩ for Checkout), Manage Inventory (Admin)

4 Entity Relationship Diagram (ERD)

Definition

An **Entity-Relationship Diagram (ERD)** is a graphical model that represents the *entities*, *attributes*, and *relationships* in a database. It is a conceptual tool used during database design to communicate structure.

4.1 Components

4.1.1 Entities

Definition

An **entity** represents a real-world object or concept about which data is stored. In a relational database, an entity maps to a *table*.

- **Strong Entity**: Exists independently; has its own primary key. Drawn as a *rectangle* (Chen) or a box (Crow's Foot).
- **Weak Entity**: Cannot exist without a related strong entity; identified by a *partial key* plus the parent's key. Drawn as a *double rectangle* in Chen notation.

Example

In a university DB: `Student`, `Course`, `Department` are strong entities. `Enrollment` may be a weak entity dependent on both `Student` and `Course`.

4.1.2 Attributes

Definition

An **attribute** is a property or characteristic of an entity or relationship. In Chen notation, attributes are drawn as *ovals* connected to their entity.

Attribute Type	Description
Simple	Atomic, cannot be divided further (e.g., `age`, `salary`).
Composite	Can be subdivided (e.g., `fullName` → `firstName`, `lastName`).
Single-valued	Holds one value (e.g., `dateOfBirth`).
Multi-valued	Can hold multiple values (double oval in Chen); e.g., `phoneNumbers`.
Derived	Computed from other attributes (dashed oval); e.g., `age` from `dateOfBirth`.
Key Attribute	Uniquely identifies an entity (underlined in Chen notation); e.g., `studentID`.

4.1.3 Relationships

Definition

A **relationship** represents an association between two or more entities. In Chen notation, a relationship is drawn as a *diamond*. Relationships can also have attributes.

- **Binary**: Between two entities (most common).
- **Ternary**: Involving three entities simultaneously.
- **Recursive / Unary**: An entity related to itself (e.g., `Employee` *manages* `Employee`).

4.2 Mapping Cardinalities

Definition

Cardinality specifies the *number of instances* of one entity that can be associated with instances of another entity through a relationship.

Cardinality Types

One-to-One (1:1)
Each instance of entity A is associated with at most one instance of entity B, and vice versa.
Example: A `Person` has one `Passport`; a `Passport` belongs to one `Person`.

One-to-Many (1:N)
One instance of entity A can be associated with many instances of entity B, but each B instance associates with at most one A.
Example: One `Department` has many `Employees`; each `Employee` belongs to one `Department`.

Many-to-One (N:1)
Many instances of entity A are associated with one instance of entity B. (Reverse perspective of 1:N.)
Example: Many `Orders` are placed by one `Customer`.

Many-to-Many (M:N)
Many instances of A associate with many instances of B, and vice versa. Implemented using a *junction/associative table*.
Example: A `Student` enrolls in many `Courses`; a `Course` has many `Students`.

4.3 Participation Constraints

Definition

A **participation constraint** specifies whether all or only some entities participate in a relationship.

Total Participation

Every instance of the entity must participate in the relationship. Also called *mandatory participation*.
In Chen notation: drawn with a **double line** connecting entity to relationship.
Example: Every `Employee` *must* work for a `Department` (total participation of Employee).

Partial Participation

Some (not all) instances of the entity may participate. Also called *optional participation*.
In Chen notation: drawn with a **single line**.
Example: Not every `Employee` manages a `Department` (partial participation of Employee in *manages*).

4.4 ERD Notations

4.4.1 Chen Notation

Developed by Peter Chen (1976). Uses distinct geometric shapes:

Symbol	Meaning
Rectangle	Entity
Double Rectangle	Weak Entity
Diamond	Relationship
Double Diamond	Identifying Relationship (for weak entity)
Oval	Attribute
Double Oval	Multi-valued Attribute
Dashed Oval	Derived Attribute
Underlined text	Key Attribute (primary key)
Double Line	Total Participation
Single Line	Partial Participation
1, M, N	Cardinality labels on relationship lines

4.4.2 Crow's Foot Notation

A more compact, widely used notation in database tools (e.g., MySQL Workbench, Lucidchart, ERDPlus). Uses line-end symbols to express cardinality and participation together:

Symbol	Meaning
`|` **(single bar)**	Exactly one
`O` **(circle)**	Zero
Crow's foot (<)	Many
`||` **(double bar)**	One and only one (mandatory one)
`O|` **(circle + bar)**	Zero or one (optional one)
`O<` **(circle + crow)**	Zero or many (optional many)
`|<` **(bar + crow)**	One or many (mandatory many)

Example

Reading Crow's Foot: In an order system, `Customer |<---O< Order` reads as: one Customer places zero-or-many Orders; each Order belongs to exactly one Customer. This is a 1:N relationship with total participation from Order side.

Note

Crow's Foot is preferred in **physical** and **logical** ERD modelling for its compactness. Chen notation is preferred in **academic** and **conceptual** design for its explicit clarity.

5 Design Patterns

Definition

Design Patterns are reusable, general solutions to commonly occurring problems in software design. They are not finished code but rather *templates* or *blueprints* that can be applied to many contexts.

Introduced by the Gang of Four (GoF) — Gamma, Helm, Johnson, Vlissides (1994) — patterns are classified into three categories:

Category	Purpose	Examples
Creational	Object creation mechanisms	Singleton, Factory, Builder, Prototype
Structural	Composition of classes/objects	Adapter, Decorator, Composite
Behavioural	Communication between objects	Observer, Strategy, Command, Iterator

5.1 Singleton Design Pattern

Definition

The **Singleton Pattern** is a *creational* design pattern that ensures a class has **only one instance** and provides a *global point of access* to that instance.

Problem it solves: When exactly one object is needed to coordinate actions across the system — e.g., a configuration manager, connection pool, logging service, thread pool.

Intent: Restrict instantiation of a class to a single object while providing global access.

5.1.1 Key Characteristics

- Private constructor (prevents external instantiation).
- A private static field holding the sole instance.
- A public static method (`getInstance()`) that returns the instance (creating it on first call).

5.1.2 Implementations

Listing 12: Eager Initialisation Singleton

```java
public class ConfigManager {
    // Instance created at class load time (thread-safe, eager)
    private static final ConfigManager INSTANCE = new
        ConfigManager();

    private String dbUrl;
    private int    maxConnections;

```

```java
    // Private constructor prevents instantiation
    private ConfigManager() {
        this.dbUrl          = "jdbc:mysql://localhost:3306/mydb";
        this.maxConnections = 10;
    }

    public static ConfigManager getInstance() {
        return INSTANCE;
    }

    public String getDbUrl()          { return dbUrl; }
    public int    getMaxConnections(){ return maxConnections; }
}

// Usage
ConfigManager config = ConfigManager.getInstance();
System.out.println(config.getDbUrl());
```

Listing 13: Lazy Initialisation with Double-Checked Locking (Thread-Safe)

```java
public class Logger {
    // volatile ensures visibility across threads
    private static volatile Logger instance;

    private Logger() { /* initialise logger */ }

    public static Logger getInstance() {
        if (instance == null) {                 // First check (no lock)
            synchronized (Logger.class) {
                if (instance == null) {         // Second check (with lock)
                    instance = new Logger();
                }
            }
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}
```

Listing 14: Bill Pugh Singleton (Best Practice using Inner Static Class)

```java
public class DatabasePool {
    private DatabasePool() { /* initialise pool */ }

    // Inner static class -- not loaded until getInstance() is called
    private static class Holder {
```

```java
        private static final DatabasePool INSTANCE = new
            DatabasePool();
    }

    public static DatabasePool getInstance() {
        return Holder.INSTANCE;    // thread-safe without
            synchronisation
    }
}
```

5.1.3 Singleton: Advantages & Disadvantages

	Description
Advantages	
Single instance	Guarantees exactly one object exists.
Global access	Easy to access from anywhere in the codebase.
Lazy init	Can defer creation until first use (save resources).
Disadvantages	
Global state	Introduces hidden global state, making code harder to test.
Tight coupling	Classes that use Singleton are tightly coupled to it.
Concurrency	Naive implementations are not thread-safe.
Testing	Difficult to mock or substitute in unit tests.

Note

Singletons are sometimes considered an **anti-pattern** in modern software design because they make unit testing difficult and hide dependencies. Prefer *Dependency Injection* where possible.

5.2 Factory Design Pattern

Definition

The **Factory Pattern** is a *creational* design pattern that provides an interface for **creating objects** but lets subclasses or configurations **decide which class to instantiate**. The factory method replaces direct calls to `new`.

Problem it solves: When the exact type of object to create is determined at runtime, or when object creation logic should be centralised and decoupled from the client code.

5.2.1 Simple Factory (Utility / Static Factory)

Listing 15: Simple Factory Example

```java
// Product Interface
public interface Notification {
    void send(String message);
}

```

```java
// Concrete Products
public class EmailNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Sending EMAIL: " + message);
    }
}

public class SMSNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Sending SMS: " + message);
    }
}

public class PushNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Sending PUSH: " + message);
    }
}

// Simple Factory -- centralises creation logic
public class NotificationFactory {
    public static Notification create(String type) {
        return switch (type.toUpperCase()) {
            case "EMAIL" → new EmailNotification();
            case "SMS"   → new SMSNotification();
            case "PUSH"  → new PushNotification();
            default → throw new IllegalArgumentException("
               Unknown type: " + type);
        };
    }
}

// Client code -- doesn't know about concrete classes
Notification n = NotificationFactory.create("EMAIL");
n.send("Your order has shipped!");
```

5.2.2 Factory Method Pattern (GoF)

The true GoF pattern defines an *abstract creator* with a factory method that subclasses override to produce specific products.

Listing 16: Factory Method Pattern

```java
// Abstract Creator
public abstract class Dialog {
    // Factory Method -- subclasses decide which Button to create
    public abstract Button createButton();

    // Template method that uses the factory method
```

```java
    public void render() {
        Button btn = createButton();
        btn.onClick();
        btn.render();
    }
}

// Concrete Creators
public class WindowsDialog extends Dialog {
    @Override
    public Button createButton() { return new WindowsButton(); }
}

public class WebDialog extends Dialog {
    @Override
    public Button createButton() { return new HTMLButton(); }
}

// Product Interface
public interface Button {
    void onClick();
    void render();
}

// Concrete Products
public class WindowsButton implements Button {
    @Override public void onClick() { System.out.println("Windows click"); }
    @Override public void render()  { System.out.println("Render Windows button"); }
}

public class HTMLButton implements Button {
    @Override public void onClick() { System.out.println("HTML click"); }
    @Override public void render()  { System.out.println("Render HTML button"); }
}
```

5.2.3 Abstract Factory Pattern

An *extension* of Factory Method — produces *families* of related objects without specifying concrete classes.

Listing 17: Abstract Factory Example

```java
// Abstract Factory
public interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

```

```java
// Concrete Factories
public class WindowsFactory implements GUIFactory {
    @Override public Button   createButton()   { return new WindowsButton();   }
    @Override public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}

public class MacFactory implements GUIFactory {
    @Override public Button   createButton()   { return new MacButton();   }
    @Override public Checkbox createCheckbox() { return new MacCheckbox(); }
}

// Client
public class Application {
    private Button   btn;
    private Checkbox chk;

    public Application(GUIFactory factory) {
        this.btn = factory.createButton();
        this.chk = factory.createCheckbox();
    }
}

// Usage
GUIFactory factory = isWindows() ? new WindowsFactory() : new MacFactory();
Application app = new Application(factory);
```

5.2.4 Factory Pattern: Advantages & Disadvantages

	Description
Advantages	
Decoupling	Client is decoupled from concrete product classes.
Open/Closed	Add new products without modifying existing client code.
Single Responsibility	Creation logic isolated in factory.
Disadvantages	
Complexity	Introduces extra classes and abstractions.
Indirection	Harder to trace which object is actually created.

Factory Pattern vs Singleton — Quick Comparison

	Singleton	**Factory**
Category	Creational	Creational
Purpose	One instance only	Create objects of varying types
Instance count	Exactly one	Multiple instances of related types
Flexibility	Low (fixed class)	High (extensible)
Use case	Config, Logger, Pool	Shape, Notification, UI components

Quick Revision Summary

SDSE Mid-Semester – Key Terms at a Glance

Term	One-Line Definition
SDLC	Structured process to plan, build, test, and maintain software.
Quality Attribute	Non-functional requirement (performance, security, reliability...).
Class	Blueprint defining structure and behaviour of objects.
Object	Concrete instance of a class with its own state.
Constructor	Initialises an object to a valid state upon creation.
Invariant	Condition always true for every instance of a class.
Encapsulation	Bundle data + methods; restrict direct field access.
Getter/Setter	Methods to read/write private fields with optional validation.
Abstraction	Hide implementation; expose only essential features.
Abstract Class	Partial template; cannot be instantiated; may have abstract methods.
Interface	Contract of methods a class must implement; enables multiple inheritance.
Inheritance	Derive attributes/methods from a parent class (IS-A).
Polymorphism	One interface, many forms; correct method resolved at runtime.
Composition	Strong HAS-A; component lifetime tied to owner.
DTO	Plain data carrier between layers; no business logic.
Observability	Understand system state via Logs, Metrics, and Traces.
Class Diagram	Static UML diagram showing classes and their relationships.
Sequence Diagram	Behavioural UML showing time-ordered message passing.
Use Case Diagram	Shows actors and system functions (what, not how).
ERD	Conceptual data model: entities, attributes, relationships.
Cardinality	Number of entity instances in a relationship (1:1, 1:N, M:N).
Total Participation	Every entity instance must participate in the relationship.
Partial Participation	Some entity instances may not participate.
Crow's Foot	Compact ERD notation using line-end symbols for cardinality.
Chen Notation	Academic ERD notation using rectangles, diamonds, ovals.
Singleton	Ensures a class has exactly one global instance.
Factory	Centralises object creation; decouples client from concrete types.

End of SDSE Mid-Semester Notes — Good luck with your exam!